



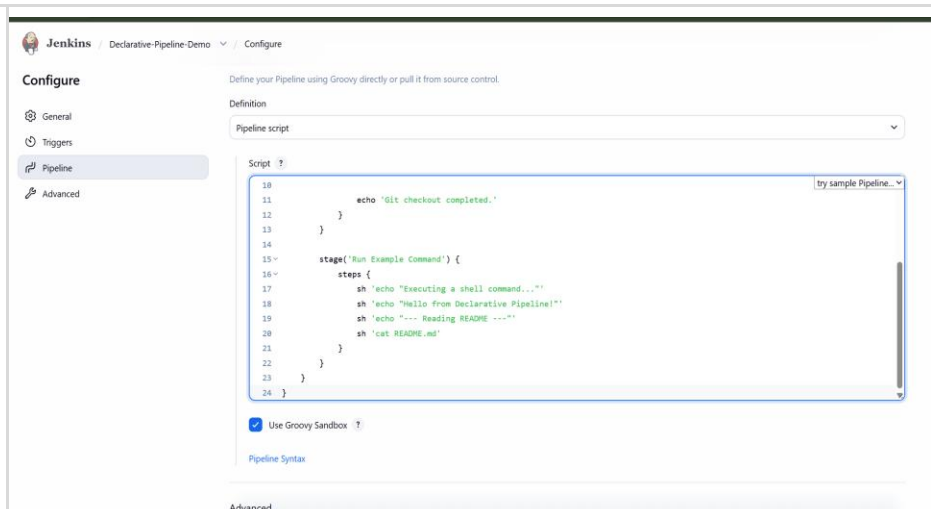
SYMBIOSIS INSTITUTE OF TECHNOLOGY (SIT)

Constituent of Symbiosis International (Deemed University), Pune

(Established under Section 3 of the UGC Act of 1956 vide notification number F-9-12/2001-U-3 of the Government of India)
Re-Accredited by NAAC with 'A' Grade

Department of Computer Science and Engineering
DevOps Lab - L3

Lab Assignment No: 6	
Name of Student	Aryan Sheladia
PRN	23070122202
Batch	2023 - 2027
Class	CSE – B1
Academic Year & Semester	2026 - 2027, 7th Semester
Title of Assignment	Basic Declarative Pipeline
Implementation & Theory	nkins / All / New Item New Item Enter an item name Declarative-Pipeline-Demo Select an item type  Pipeline Build, test, and deploy using pipelines. Supports stages, parallel work, and running on multiple agents.  Freestyle project Classic, general-purpose job type that checks out from up to one SCM, executes build steps serially, followed by post-build steps like archiving artifacts and sending email notifications.  Multi-configuration project Suitable for projects that need a large number of different configurations, such as testing on multiple environments, platform-specific builds, etc.  Folder Creates a container that stores nested items in it. Useful for grouping things together. Unlike view, which is just a filter, a folder creates a separate namespace, so you can have multiple things of the same name as long as they are in different folders. Create Pipeline project — Created a new Jenkins item named Declarative-Pipeline-Demo and selected Pipeline as the project type. This creates a Pipeline-as-Code job for defining and executing a Declarative Pipeline.



Define pipeline script — Configured the Pipeline Definition as **Pipeline script** and created a Declarative Pipeline with two stages: **Checkout Source Code**, which retrieves the Git repository, and **Run Example Command**, which executes shell commands and displays the README contents.

Declarative Pipeline-Demo / #1 / Pipeline Steps

Step	Arguments	Status
Start of Pipeline - (12 sec in block)		✓
node - (11 sec in block)		✓
node block - (11 sec in block)		✓
stage - (9.8 sec in block)	Checkout Source Code	✓
stage block (Checkout Source Code) - (9.7 sec in block)		✓
git - (9.6 sec in self)		✓
echo - (22 ms in self)	Git checkout completed.	✓
stage - (1.3 sec in block)	Run Example Command	✓
stage block (Run Example Command) - (1.2 sec in block)		✓
sh - (0.32 sec in self)	echo "Executing a shell command..."	✓
sh - (0.3 sec in self)	echo "Hello from Declarative Pipeline!"	✓
sh - (0.29 sec in self)	echo "---- Reading README ----"	✓
sh - (0.29 sec in self)	cat README.md	✓

Pipeline Stage View — Executed Build #1 successfully and verified that both pipeline stages, **Checkout Source Code** and **Run Example Command**, completed without errors.

Declarative-Pipeline-Demo / #1

```
## Repo structure

...text
.
├─ Assignment - 1/  Git setup & basic workflow
├─ Assignment - 2/  Git branching & merging
├─ Assignment - 3/  Git collaboration & conflict resolution
├─ Assignment - 4/  Jira project & issue tracking
├─ Assignment - 5/  Jenkins setup & freestyle project
├─ Assignment - 6/  Basic declarative pipeline
├─ Assignment - 7/  Pipeline with parameters
├─ Assignment - 8/  Dockerfile & image build (Flask app)
├─ Assignment - 9/  Container management & networking
├─ Assignment - 10/ Docker volumes & data persistence
├─ Assignment - 11/ Docker Compose (Flask + Redis)
├─ Assignment - 12/ Kubernetes Pod & Deployment
├─ Assignment - 13/ Kubernetes Service & networking
├─ Assignment - 14/ CI/CD: Jenkins pipeline building Docker image
├─ Assignment - 15/ Terraform basics (local provider)
└─ README.md
...

[Pipeline] }
[Pipeline] // stage
[Pipeline] }
[Pipeline] // node
[Pipeline] End of Pipeline
Finished: SUCCESS
```

Console Output verification — Verified the execution logs of Build #1, confirming successful Git checkout, execution of the shell commands, and completion of both pipeline stages. The build ended with Finished: SUCCESS.